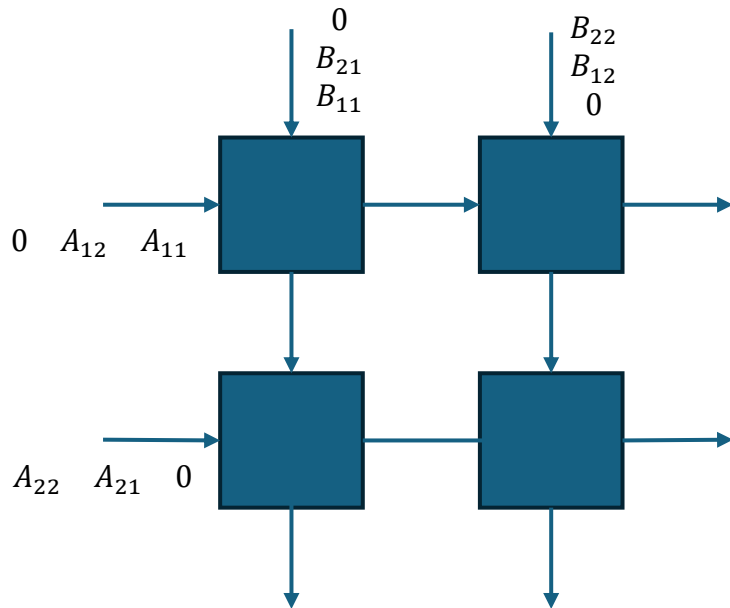


Tiling for Systolic Arrays

- A **systolic array** is a parallel computing hardware structure of connected processing elements (cells) that rhythmically process and transmit data
- TPUs** use systolic arrays as their core computational component to efficiently perform the high-throughput matrix multiplication required for AI workloads



A "tile" is a pre-defined, fixed-size chunk of a larger matrix.

$$A \times B = C$$

A_{11}	A_{12}	A_{13}	0
A_{21}	A_{22}	A_{23}	0
A_{31}	A_{32}	A_{33}	0
0	0	0	0

B_{11}	B_{12}	B_{13}	0
B_{21}	B_{22}	B_{23}	0
B_{31}	B_{32}	B_{33}	0
0	0	0	0

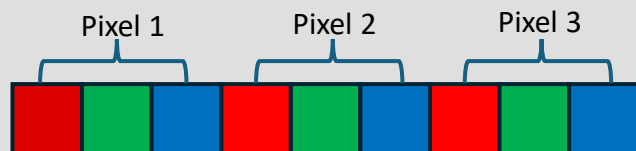
Both Matrices A and B are partitioned into fixed-size tiles that match the array's row bandwidth. Zeros pad the boundary, so every tile is the same size and enters on the same rhythm. As each A tile streams to the right and each B tile streams to the bottom, its values are reused across multiple columns of compute cells, letting one load of A and B participate in many multiplications before it exits.

Systolic 2x2 multiplication example: A tiles shift to the right and B tiles shift down. Each processing element multiplies its local A and B values and adds to a running sum. After several beats, the correct outputs emerge at the bottom/right without ever stopping the flow.

Data Layout Transformations

- N: Number of images in the batch.
- C: Number of Channels per image (e.g., 3 for Red, Green, Blue).
- H: Height of the image in pixels.
- W: Width of the image in pixels.

NHWC



- **Order:** Batch, Height, Width, Channels
- **What it means:** For each pixel, all its color/feature values are stored next to each other.
- **Typical:** TPUs/mobile NPUs; kernels that combine channels at a pixel
- Order in the above image: pixel 1(red)-> pixel 1 (green), pixel 1 (blue), pixel 2(red), pixel

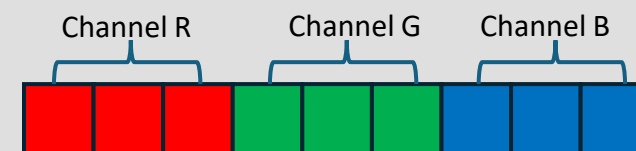
Reorder/transpose axes



Example permutation: [0, 3, 1, 2]
turns NHWC into NCHW

*No data changes—just the order
we store it.*

NCHW



- **Order:** Batch, Channels, Height, Width
- **What it means:** For each channel, all the pixels in the image are stored next to each other.
- **Good when:** You sweep over images one channel at a time.
- **Typical:** Convolution on many GPUs

For large matrix multiplies, we often go beyond NHWC/NCHW and use packed tile layouts that match the GEMM kernel

GEMM stands for General Matrix Multiplication. It's the mathematical name for the core operation that powers deep learning and scientific computing: multiplying two large matrices together.

Tensor Processing Units (TPUs)

Google's Specialized AI Architecture

Systolic Array Design:

TPUs implement large 2D arrays of processing elements where data flows rhythmically through the grid:

```
// Conceptual systolic array operation
for (int cycle = 0; cycle < matrix_size; cycle++) {
    // Each PE performs: accumulator += input_from_top * weight_from_left
    // Data flows: inputs downward, weights rightward, outputs collected
    PE[i][j].accumulate(input_from_above[i], weight_from_left[j]);

    // Shift data for next cycle
    shift_inputs_down();
    shift_weights_right();
}
```

TPU Characteristics:

- **Large Matrix Units:** 128×128 or 256×256 systolic arrays
- **Mixed Precision:** BF16 computation with FP32 accumulation
- **Software Stack:** XLA compiler for automatic optimization
- **High Throughput:** Optimized for training large models

Design Trade-offs:

- Excellent for large matrix operations
- Less flexible than GPUs for irregular computations
- Requires compiler optimization for efficiency



Google's Tensor Processing Unit (TPU)
Image Source: unwire.pro

Systolic Array Deep Dive

Understanding Specialized Matrix Computation

Systolic Array Principles:

Data flows through a 2D grid of processing elements in a synchronized, rhythmic pattern

Advantages:

- **High Utilization:** All PEs active simultaneously during steady state
- **Predictable Data Flow:** No complex routing or scheduling
- **Scalability:** Linear scaling with array size
- **Energy Efficiency:** Minimal control overhead

Limitations:

- **Matrix Size Constraints:** Optimal for specific dimension ranges
- **Setup Overhead:** Filling pipeline takes time
- **Limited Flexibility:** Primarily optimized for dense matrix operations

Applications:

- Convolutional layers (after im2col transformation)
- Fully connected layers
- Attention mechanisms in transformers

Reminder from Week 3: Systolic Arrays use Tiling

```
// PE (Processing Element) operation each cycle
class ProcessingElement {
    int32_t accumulator;

    void cycle(int8_t input_from_north, int8_t weight_from_west) {
        // Multiply-accumulate operation
        accumulator += input_from_north * weight_from_west;

        // Pass data to neighbors
        send_to_south(input_from_north);
        send_to_east(weight_from_west);
    }
};
```